Fine■tuning a MACE MLIP using PySCF — Detailed Practical Guide

This document expands the guidance from our chat into a more detailed, practical workflow, and ends with a concise TL;DR checklist you can follow quickly.

1) Big Picture: What you are actually doing

You are training (or fine■tuning) a MACE interatomic potential so that it reproduces reference quantum■chemistry data computed with PySCF. The model learns a mapping from atomic positions and species → energies and forces. PySCF acts as the 'teacher' that generates accurate labels; MACE becomes the fast surrogate used for MD, relaxations, or sampling.

Core loop: generate structures → compute PySCF labels → train/fine■tune MACE → test → add hard cases (active learning) → retrain.

2) Choose the training target: Direct learning vs Δ■learning

Direct learning (simplest): train MACE to predict PySCF energies and forces directly.

Δ■**learning** (often more data■efficient): train a correction model that learns the difference between a baseline potential and PySCF.

Formula: ΔE = E(PySCF) − E(base), and similarly for forces. Final prediction = base + correction.

When to pick which: if you already have a decent general MACE model or another potential, Δ■learning usually converges faster and is more stable. If not, start with direct learning.

3) Build a good geometry dataset (most important step)

Models fail mainly because the dataset lacks coverage. You want examples of configurations the model will actually see during use.

- Near■equilibrium geometries (optimized structures).
- Small random distortions (bond lengths, angles, lattice strain).
- Finite■temperature snapshots from short MD runs.
- Reaction or transition■like geometries if chemistry changes matter.
- Edge cases (compressed/expanded structures) so the model does not extrapolate wildly.

Rough scale: hundreds–thousands of structures for a small molecular domain; thousands–tens of thousands for broader chemistry or materials.

4) Label data with PySCF

For each geometry, run a single consistent PySCF setup (same method, basis, charge, spin, and convergence settings). Store at least: total energy and nuclear gradients (forces = −gradients).

Unit conversion (critical):

PySCF energies are in Hartree; gradients are in Hartree/Bohr.

MACE/ASE typically expects eV and eV/Å.

```
1 Hartree = 27.211386245988 eV
1 Bohr = 0.529177210903 Å
Convert forces: (Hartree/Bohr) × (27.211386245988 / 0.529177210903) → eV/Å
```

Make sure sign conventions are correct: if PySCF returns gradients, forces are negative gradients.

5) Store data in an MLIP■friendly format

A common path is ASE Atoms objects with per■structure fields: energy, forces, and optional stress/virial. Formats like extended XYZ, .traj, or HDF5 are commonly used.

Consistency matters more than format: identical atom ordering, units, and metadata across the dataset.

6) Fine■tuning strategy for MACE

Key training choices:

- Loss usually combines energy + forces; forces get a much larger weight.
- Use a lower learning rate for fine■tuning than training from scratch.
- Start by training only the output head, then unfreeze deeper layers.
- Monitor validation energy MAE (eV/atom) and force RMSE (eV/Å).
- Keep a held■out validation set that includes difficult geometries.

Practical two■phase recipe:

- Phase A: freeze most layers, train output head for stability (short run).
- Phase B: unfreeze more layers and continue with smaller learning rate.

7) Active learning (how strong MLIPs are actually built)

After initial training, run simulations using MACE and search for configurations where it is unreliable. Those structures are then labeled with PySCF and added back into training.

- Run short MD or relaxation trajectories with the current model.
- Select frames with large forces, unstable behavior, or model disagreement.
- Recompute those frames with PySCF.
- Append to dataset and retrain.

This loop typically gives better robustness than just adding random data.

8) PySCF-specific pitfalls

- SCF convergence can fail on distorted geometries; add retries or fallback settings.
- Keep charge and spin consistent across all samples.
- For periodic systems (pyscf.pbc), be careful with k-points and basis choices.
- Filter out non-converged calculations from the dataset.

9) What “fine■tuning” usually means in practice

- From scratch: train MACE only on your PySCF data.
- Domain adaptation: start from a pretrained MACE checkpoint and fine■tune on your PySCF set.
- Δ■model: keep a base potential and train only the correction to PySCF.

TL;DR — Practical Checklist

- Generate diverse geometries (equilibrium + distortions + MD snapshots).
- Compute PySCF energies and gradients with ONE consistent method/basis.
- Convert units to eV and eV/Å; forces = –gradients.
- Store data in ASE-compatible format (energy + forces required).
- Fine■tune MACE with strong force weighting and a small learning rate.
- Validate on held■out difficult structures.
- Run MACE simulations, collect failure cases, relabel with PySCF.
- Retrain — repeat active learning until errors stabilize.

One■sentence summary: Use PySCF to label a diverse set of geometries with energies and forces, fine■tune MACE on that data (forces weighted heavily), and iteratively improve robustness via active learning.